

Documentatie – Proiect POO

ROCK the SHOP

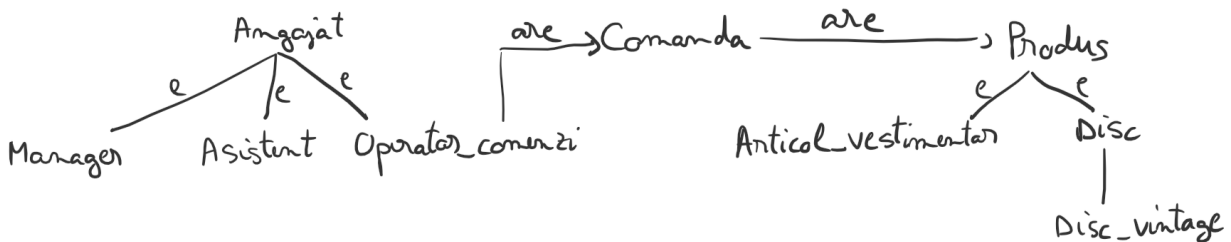
Sistem pentru administrarea unui magazin

Martin Razvan-Stefan

Descriere proiect:

Un magazin online specializat in produse rock comercializeaza diverse categorii de produse si gestioneaza activitati precum primirea, procesarea și livrarea comenzilor. Proiectul include un program scris în C++ care faciliteaza urmatoarele operatiuni: gestionarea angajatilor magazinului, administrarea stocului de produse, procesarea comenzilor precum si generarea de rapoarte bazate pe comenzile procesate.

Descrierea claselor folosite in implementare:



Pentru a tine evidenta angajatilor am implementat clasa de baza Angajat cu urmatoarele attribute: nume, prenume, ID, CNP, toate de tip string si data de angajare pe care am retinut-o intr-o variabila de tip struct tm din biblioteca ctime. ID-ul este unic, atribuit la crearea obiectelor si este obtinut cu ajutorul unei variabile statice ce retine nr obiectelor create (ex: ang3 – ID-ul celui de-al treilea obiect creat). De asemenea, la crearea unui obiect se verifica urmatoarele conditii: numele si prenumele angajatului trebuie sa aiba minim 3 caractere si maxim 30, CNP-urile trebuie sa fie valide, data de angajare sa fie una posibila, iar angajatul trebuie sa aiba minim 18 ani pentru a se putea angaja. Daca macar una dintre conditiile mentionate mai sus nu este respectata, programul arunca o exceptie, afiseaza motivul, iar obiectul nu este creat.

Magazinul are 3 tipuri de angajati: Manager, Asistent si Operator de comenzi, clase derivate din clasa Angajat. Din cauza modului de calcul diferit al salariului in functie de tipul de angajat, am folosit o functie virtuala supraincarcata in fiecare clasa derivata. Pentru afisarea datelor unui angajat si citirea de la tastatura am implementat functie de afisare (ce permite si afisarea in fisier), respectiv operator de citire in clasa Angajat, deoarece in clasele derivate nu exista attribute suplimentare.

Pentru a tine evidenta produselor din stoc am implementat clasa de baza Produs cu attributele, string ID, denumire, int stoc, double pret de baza. ID-ul este generat asemanator ca la angajati folosind un atribut static la creare pentru a-i garanta unicitatea (ex prod5 – ID-ul celui de-al 5-lea produs creat). Stocul si pretul trebuie sa fie numere prozitive, altfel se arunca exceptii si nu se creeaza obiectul.

Magazinul are 3 tipuri diferite de produse: articole vestimentare, discuri si discuri vintage, folosite in program astfel:

Clasa Articol_vestimentar este derivata din clasa Produs si are in plus attributele, string culoare, marca.

Clasa Disc este derivate tot din clasa Produs si are in plus attributele, string casa discuri, trupa, nume album, un bool, care are valoarea 1 pt CD si 0 pt vinil si un struct tm din ctime pentru a stoca data punerii in vanzare a produsului. De asemenea, si aici data trebuie sa fie valida altfel se arunca exceptii.

Clasa Disc_vintage este derivata din clasa Disc si are in plus un bool ce are valoarea 1 daca discul e mint si 0 daca nu e si un coeficient de raritate, numar intreg intre 1 si 5 (in caz contrar se arunca exceptii).

Pentru a putea afla costul final, ce include impachetarea si livrarea, al fiecarui produs diferit am folosit o functie virtuala din cauza modului diferit de calcul al acestora. De asemenea, am implementat o functie virtuala de afisare, ce permite si afisarea datelor in fisiere si o functie virtuala de citire, ce permite atat citirea de la tastatura cu mesaje ajutatoare pe ecran (ex: Introduceti denumirea produsului:), cat si citirea din fisere fara acestea.

Pentru a procesa comenzi am creat clasa Comanda ce are ca attribute, un string ID (creat ca in cazul claselor Angajat si Produs, ex: com2), data de primire stocata tot intr-o variabila struct tm, durata comenzii in secunde, timpul ramas din comanda (initial are aceasi valoare cu durata comenzii) si o lista de adrese de produse, list<Produs*> produse, pentru a putea stoca oricare dintre cele 3 tipuri de produse diferite (atributul din clasa Produs pentru stoc in cazul comenzilor devine numarul de bucati din acest produs dorite de client). Din cauza celei din urma, in clasa comanda, a trebuit sa implementez si constructor de copiere, operator de atribuire si destructor, acestea ne generandu-se correct. Pentru a face acest lucru a trebuit sa creez o metoda virtuala in clasa Produs (supraincercata in toate clasele derivate) pentru a putea copia profund obiectele, in caz contrar destructorul din Produs ar fi incercat sa elibereze de 2 ori aceasi memorie, ceea ce ar fi dus la un crash al aplicatiei.

Tot in clasa Comanda am implementat functii pentru verificarea validitatii unei comenzi. Pentru ca o comanda sa fie valida si sa poata fi procesata, toate produsele din comanda trebuie sa existe in magazin, daca exista sa fie suficiente bucati in stoc din acel fel, pretul total al tuturor produselor din comanda sa fie de minim 100 de lei fara a lua in calcul costurile de impachetare si livrare, iar comanda sa aiba maxim 3 articole vestimentare si maxim 5 discuri (in acest caz se iau in calcul si nr de bucati cerute din fiecare tip de produs). In caz ca una dintre conditii nu este indeplinita comanda nu poate fi procesata, nu este atribuita unui operator de comenzi si se afiseaza un mesaj cu motivul. Pentru cautarea produselor din comanda in stocul de produse al magazinului, pentru

verificarea existentei acestora, am implementat un operator virtual de egalitate in clasa Produs, supraincarcat in toate clasele derivate (pentru ca 2 produse sa fie egale toate attributele acestora trebuie sa fie identice in afara de int stoc). De asemenea, am supradefinit operatorul de citire si operatorul de afisare in clasa Comanda, ce folosesc functile virtuale respective din clasa Produs.

Gestiunea Angajatilor:

Pentru crearea evidentei de anajati, pentru a putea stoca obiecte din clasele Manager, Asistent si Operator_comenzi in acelasi loc, am folosit o lista de adrese de angajati, `list<Angajat*> evid_ang`, ce a fost populate cu ajutorul constructorilor cu parametrii din clasele respective.

Folosind aceasta evidenta a angajatiilor, programul permite adaugarea unui angajat nou in evidenta folosind functiile virtuale de citire din Angajat supraincarcata in clasele derivate (data de angajare al unui angajat adaugat nou se considera data curenta), stergerea unui angajat din evidenta, modificarea numelui unui angajat cu un nume dat de la tastatura, afisarea tuturor datelor despre un singur angajat. Cautarea angajatului pentru stergere/modificare/afisare se face dupa nume si prenume. In cazul in care angajatul cautat nu exista in evidenta, functiile afiseaza mesaje in acest sens si permit introducerea repetata a angajatului cautat pentru diferitele operatii. De asemenea, programul permite afisarea tuturor evidentei de angajati cu toate datele fiecaruia, inclusiv salariul din luna curenta (luna curenta este luata in functie de data actuala a sistemului). Pentru verificarea numarului minim de angajati necesari pentru functionarea magazinului am folosit un vector static de dimensiune 3, astfel: pe prima pozitie a acestuia se afla nr de obiecte de tip Manager create, pe a doua nr de obiecte de tip Disc, iar pe a treia cele de tip Disc_vintage. La incheierea programului se apeleaza o functie pt eliberarea memoriei ocupata de lista de angajati.

Gestiunea Stocului de Produse:

Pentru crearea stocului de produse, pentru a putea stoca produse de tipul tuturor claselor derivate din Produs, am folosit o lista de adrese de produse, `list<Produs*> stoc_prod`, populata cu ajutorul constructorilor cu parametrii din fiecare clasa.

Folosind stocul de produse, programul permite adăugarea de produse noi în stoc, (dacă se introduce un produs deja existent acesta nu se adaugă la finalul listei ci i se modifica doar stocul celui deja existent), ștergerea de produse din stoc, modificarea cantitatii stocului unui anumit produs, afișarea datelor despre un anumit produs. Căutarea produsului pentru efectuarea operațiilor de stergere/modificare/afișare al unui produs se face după ID-ul produsului. Daca produsul cautat nu exista in stoc, se afiseaza un mesaj in acest sens si se poate incerca din nou cautarea. De asemenea, programul permite afișarea datelor pentru toate produsele din stoc la cerere. Pentru verificarea numărului minim de produse necesare pentru funcționarea magazinului am folosit, ca pt angajați, un vector static ce conține nr de obiecte create din fiecare tip.

Procesarea Comenzilor:

Comenzile sunt citite din fisierul cu numele: comenzi.in. Pe prima linie a acestuia se afla nr de comenzi din fisier. Apoi fiecare comanda este scrisa dupa cum urmeaza in exemplul de mai jos:

```
5 8 2024 20 3
2 Bohemian Rhapsody, 2 90 1 Asylum Records, 8 12 1976 Eagles, Hotel California,
1 Rochie, 3 350 Rosu, Zara,
3 Paint It Black, 1 400 0 Columbia Records, 18 9 1975 Bruce Springsteen, Born to Run, 1 5
```

Pe prima linie, primele 3 nr reprezinta data comenzii, urmata de durata comenzii si de nr de produse diferite din comanda, iar pe urmatoarele linii sunt efectiv produsele cerute, cu un identificator inaintea acestora pentru a stabili tipul. Fiecare comanda e separata printr-o linie libera. Dupa ce sunt citite, comenzile sunt puse intr-o coada de comenzi.

Comenziile sunt procesate de operatorii de comenzi, care nu pot avea mai mult de 3 comenzi alocate in acelasi timp. Pentru a tine evidenta a ce comenzi are asociate la un mom de timp un operator am adaugat in clasa `Operator_comenzi` un atribut `queue<Comanda>`. De asemenea, am folosit o functie ce extrage toti operatorii de comenzi din evidenta completa de angajati si ii adauga intr-o lista separata. Pt procesarea tuturor comenzilor am folosit un `while` care se opreste doar atunci cand coada de comenzi citite din fisier si toate cozile de comenzi ale tuturor operatorilor de comenzi sunt goale. La fiecare iteratie a `while`-ului se verifica daca cozile operatorilor de comenzi sunt pline, daca sunt, se afiseaza un mesaj, iar daca nu sunt, se adauga la finalul acestora o comanda din coada mare (cat timp aceasta nu e goala) si se afiseaza un mesaj cu ce comanda a fost adaugata carui operator. Dacă nu toate cozile operatorilor sunt pline atunci se adauga câte una fiecăruia. Dacă comanda curenta nu e valabila, se incearca adăugarea pana se găsește una valabila sau se golește coada de comenzi. De asemenea, la fiecare iteratie din `while` trece o secunda din durata primei comenzi a fiecărui operator de comenzi, scade variabila timp rămas din comanda respectiva. Când aceasta ajunge la 0 se da `pop` la comanda respectiva din coada si se afiseaza un mesaj cu ce comanda a fost finalizata si cine a finalizat-o. Functia mai afisează si secunda din simulare la care se afla si date despre ce contine comanda si cine o procesează atunci când operatorul respectiv începe sa o proceseze. De asemenea, am adaugat clasei `Operator_comenzi` 2 attribute, unul ce retine nr de comenzi procesate si un vector<double> ce retine valoarea tuturor comenzilor procesate de acesta. Aceste attribute se modifica corespunzător la finalizarea unei comenzi si sunt utile pentru obtinerea raporturilor. La finalul procesarii tuturor comenzilor, se updateaza toti operatorii de comenzi din lista cu evidenta tuturor angajaților, cu valoarea tuturor comenzilor procesate pentru a se putea calcula salariul corect al acestora.

Alte informatii:

Meniul permite alegerea intre 5 operatii: gestionarea angajatilor, gestionarea produselor, procesarea comenzilor, obtinerea raporturilor sau inchiderea aplicatiei. Obtiunea de obtinere a raporturilor poate fi accesata doar dupa ce au fost procesate comenzile. De asemenea, de fiecare data cand se intra in meniul principal se va verifica nr minim de angajati si nr minim de produse necesare pentru functionarea magazinului. Daca una dintre cele 2 conditii nu este indeplinita, se afiseaza nr minim de angajati/produse din fiecare tip necesare pentru ca magazinul sa poata functiona si va oferi acces doar la metoda de adaugare de noi angajati/produse sau la cea de inchidere a aplicatiei. Pentru a nu aglomera consola cu prea multe mesaje, de fiecare data cand se alege o operatie aceasta se goleste.